

گزارش پروژه ۳ درس طراحی پایگاه داده‌ها

کلاستر توزیع شده MongoDB و تحلیل متادیتای Electronics Amazon

استاد: مریم رمضانی

اعضای پروژه

محمد رضا ایزدی 402110916

کیما قدیر 402106323

پارسا نوروزی منش 402106723

۱ معرفی کوتاه پروژه

در این پروژه، متادیتای محصولات بخش Electronics از مجموعه Reviews Amazon ۲۰۲۳ در MongoDB ذخیره و تحلیل شده است. فایل اصلی داده به صورت JSONL و با حجم تقریبی 5.25 GB شامل 1,610,012 محصول است. به دلیل حجم بالای فایل، ورود داده‌ها به صورت Streaming و خطبه‌خط انجام شده و برای جلوگیری از مصرف بیش از حد حافظه، اسناد در های Batch کوچک وارد پایگاه داده شده‌اند.

لایه ذخیره‌سازی اصلی پروژه یک Set Replica سه‌نودی MongoDB با یک Primary و دو Secondary است. علاوه بر طراحی مناسب برای دسته‌بندی‌های چندسطحی و ویژگی‌های فنی پویا، پنج دسته Query الزامی، گذاری Index خروجی‌های explain()، آزمون Race، Failover Condition، زنده و ارزیابی Performance در سه حالت Standalone بدون Cluster Index، بدون Index و Cluster بهینه با Index اجرا شده‌اند. نتایج واقعی تست‌ها و نمودارهای تولیدشده با Matplotlib در بخش‌های بعدی گزارش آورده شده‌اند.

۲ پیکربندی کلاستر MongoDB

معماری اصلی شامل سه نمونه MongoDB نسخه ۷ با نام Set Replica برابر rs0 است. نداشت نودها به پورت‌های میزبان به شکل زیر در نظر گرفته شد:

نود	نقش در حالت عادی	پورت میزبان
mongo1	Primary یا Secondary	27017
mongo2	Primary یا Secondary	27018
mongo3	Primary یا Secondary	27019

تمام نودها در یک Set Replica قرار دارند و انتخاب Primary توسط مکانیزم Election خود MongoDB انجام می‌شود؛ بنابراین هیچ نودی به صورت دائمی Master فرض نشده است. برای پایداری محیط Windows/WSL و جلوگیری از فشار بیش از حد حافظه در زمان Import و Build، Index و WiredTiger کش هر نود روی 1 GB محدود و برای هر نود Volume مستقل در نظر گرفته شد.

۱.۲ دستور مقداردهی اولیه Set Replica

پیکربندی اولیه با ساختار زیر انجام شد:

```
rs.initiate({
  _id: "rs0",
  members: [
    { _id: 0, host: "mongo1:27017" },
    { _id: 1, host: "mongo2:27017" },
    { _id: 2, host: "mongo3:27017" }
  ]
})
```

Initializer پروژه به صورت idempotent پیاده‌سازی شده است؛ یعنی اگر Set Replica قبلاً ایجاد شده باشد، اجرای مجدد سرویس Initialization باعث خطا یا مقداردهی دوباره نمی‌شود.

۲.۲ تفکیک عملیات Read و Write

در اتصال PyMongo، های Query خواندنی کلاستر با سیاست secondaryPreferred اجرا می‌شوند. در این حالت Driver در صورت وجود Secondary قابل استفاده، Read را به ها Replica می‌فرستد و در صورت در دسترس نبودن Secondary می‌تواند به Primary بازگردد. عملیات Write طبق منطق Set Replica توسط MongoDB به Primary فعلی هدایت می‌شوند.

در Benchmark حالت، Preference Read Standalone برابر primary است؛ اما در دو حالت کلاستر، Prefer-Read در برابر secondaryPreferred استفاده شده است. این انتخاب باعث می‌شود بار های Query خواندنی از Primary جدا شود و رفتار مورد انتظار پروژه برای تقسیم بار Read/Write پیاده‌سازی شود.

۳ آزمون Failover زنده

برای بررسی تحمل خطا، Primary فعلی به صورت عمدی متوقف شد و وضعیت Set Replica در سه مرحله ثبت گردید.

مرحله	mongo1	mongo2	mongo3
قبل از قطع Primary	Secondary	Secondary	Primary
بعد از توقف mongo3	Primary	Secondary	Health / Down
پس از بازگشت mongo3	Primary	Secondary	Secondary

در وضعیت اولیه، mongo3 نقش Primary داشت. پس از توقف این نود، MongoDB به صورت خودکار Election را انجام داد و mongo1 به Primary جدید تبدیل شد. پس از راه اندازی مجدد mongo3، این نود به کلاستر بازگشت اما نقش Primary قبلی خود را پس نگرفت و به عنوان Secondary با $health=1.0$ فعالیت کرد. بنابراین هر سه مرحله مورد نیاز Failover شامل قطع Primary، انتخاب خودکار Primary جدید و بازگشت نود قبلی به عنوان Replica با موفقیت مشاهده شد.

۴ پیش پردازش و ورود Streaming داده ها

فایل داده از نوع JSONL است؛ یعنی هر خط یک شیء JSON مستقل مربوط به یک محصول است. بارگذاری کامل فایل در حافظه برای فایلی با حجم بیش از پنج گیگابایت مناسب نیست، بنابراین پردازش به صورت خط به خط انجام شد. منطق اصلی پردازش برای هر خط شامل مراحل زیر است:

- خواندن یک خط از فایل بدون بارگذاری کل فایل در RAM؛
- تبدیل رشته JSON به Document؛
- اعمال تبدیل Schema روی categories و details؛
- تعیین `_id` پایدار بر اساس `parent_asin`؛
- افزودن Document به Batch؛
- درج Batch با `insert_many(..., ordered=False)`؛
- ذخیره Checkpoint بر اساس Offset Byte فایل برای Resume بعد از قطع یا Crash.

در نسخه Resumable، استفاده از `parent_asin` به عنوان `_id` پایدار باعث شده است Retry یک Batch پس از خطای شبکه یا Crash، داده تکراری ایجاد نکند. Duplicate هایی که ناشی از تکرار یک Batch تأیید نشده هستند قابل تشخیص و تحمل اند.

نتیجه Import نهایی به صورت زیر بود:

شاخص	مقدار
حجم فایل منبع	5,246,144,134 bytes
تعداد نهایی ها Document	1,610,012
Size Batch	1000
زمان اجرای موفق نهایی	2393.27 s
نرخ پردازش	672.72 records/s
Checkpoint نهایی	انتهای کامل فایل

۵ طراحی Schema اسناد

هر محصول به صورت یک Document مستقل ذخیره شده است. فیلدهای اصلی متادیتای آمازون حفظ شده‌اند، اما دو بخش مهم یعنی ساختار دسته‌بندی و مشخصات فنی پویا برای Query و گذاری Index بازطراحی شده‌اند.

۱.۵ نمونه ساختار نهایی Document

```
{
  "_id": "B08K7KC4GR",
  "parent_asin": "B08K7KC4GR",
  "title": "...",
  "main_category": "...",
  "average_rating": 4.5,
  "rating_number": 120,
  "price": 29.99,
  "categories": [
    "Electronics",
    "Computers",
    "Micro SD Cards"
  ],
  "details": [
    {"k": "Brand", "v": "Digi-Tattoo"},
    {"k": "Color", "v": "Fresh Marble"}
  ],
  "features": [...],
  "description": [...],
  "images": [...],
  "videos": [...],
  "store": "...",
  "bought_together": null
}
```

۲.۵ مسطح‌سازی دسته‌بندی‌ها

ساختار سلسله‌مراتبی Category به یک آرایه مرتب و بدون تکرار تبدیل شد. در نتیجه برای جستجوی یک Category لازم نیست سطح آن از قبل مشخص باشد. Query روی عضو آرایه اجرا می‌شود و تفاوتی ندارد عبارت موردنظر در ریشه، سطح میانی یا برگ نهایی مسیر قرار گرفته باشد.

این مدل نسبت به نگهداری یک Tree یا مجموعه‌ای از فیلدهای category_level_1، category_level_2، ... مزیت دارد؛ زیرا عمق دسته‌بندی در محصولات مختلف می‌تواند متفاوت باشد و Query روی یک آرایه ثابت ساده‌تر و قابل گذاری‌Index است.

۳.۵ مدل‌سازی ویژگی‌های فنی پویا

فیلد خام details شامل کلیدهایی است که از محصولی به محصول دیگر تغییر می‌کنند. نگهداری مستقیم آن به شکل Document پویا باعث می‌شود نام‌های Field قابل جستجو از قبل معلوم نباشد و ساخت یک Index عمومی برای ترکیب «نام ویژگی + مقدار ویژگی» دشوار شود. بنابراین ساختار زیر انتخاب شد:

```
"details": [  
  {"k": "Brand", "v": "Digi-Tattoo"},  
  {"k": "Color", "v": "Fresh Marble"}  
]
```

این ساختار اجازه می‌دهد یک Index Compound روی details.k و details.v ایجاد شود و های Query Q۴ و Q۵ با \$elemMatch روی یک زوج Key/Value دقیق اعمال شوند. در صورتی که مقدار یک ویژگی خود شامل List یا Object باشد، مقدار آن به String JSON پایدار تبدیل می‌شود تا Field v همچنان قابل گذاری‌Index باقی بماند.

۴.۵ طرح‌های جایگزین و دلیل انتخاب Schema فعلی

- نگهداری Category به شکل Tree خام: این روش ساختار منبع را بهتر حفظ می‌کند، اما Query «وجود Category در هر سطح» پیچیده‌تر می‌شود و گذاری‌Index روی مسیرهای با عمق متغیر دشوارتر است. آرایه مسطح برای نیاز Query پروژه مناسب‌تر است.
- نگهداری details به شکل Object پویا: خوانایی هر Document بالا است، اما کلیدهای متغیر باعث انفجار تعداد مسیرهای Field و دشواری در ایجاد Index عمومی می‌شوند. مدل k/v یک Schema یکنواخت ایجاد می‌کند.
- انتقال details به Collection جداگانه: این کار می‌تواند برای حجم بسیار بزرگ‌ها Attribute مفید باشد، اما برای واکنشی کامل محصول نیاز به \$lookup یا چند Query ایجاد می‌کند. چون ویژگی‌ها جزء ذاتی همان محصول هستند، Embed کردن آن‌ها Locality بهتری دارد.
- تبدیل تمام ویژگی‌ها به ستون‌های ثابت: به دلیل ناهمگونی شدید محصولات، Electronics این مدل تعداد زیادی Field خالی و Schema بسیار Sparse ایجاد می‌کند و توسعه آن با ویژگی‌های جدید دشوار است.

بنابراین Schema انتخابی ضمن حفظ Document محور بودن، MongoDB های Query مورد نیاز و گذاری Index روی داده‌های پویا را ساده می‌کند.

۶ پنج دسته Query عملیاتی

برای حفظ یک Workload ثابت در های Benchmark نهایی، مقادیر زیر از داده واقعی انتخاب و در هر سه حالت استفاده شدند:

Query	ورودی نهایی
Q۱	parent_asin = B08K7KC4GR
Q۲	صفحه اول، page_size=20
Q۳	Micro SD Cards
Q۴	Brand = Digi-Tattoo
AND Q۵	Color = Fresh Marble و Brand = Digi-Tattoo
OR Q۵	Publisher = Fourth Estate; First Edition (January یا Brand = Digi-Tattoo 1, 2009)

۱.۶ Q۱: واکنشی محصول بر اساس شناسه

این Query یک محصول خاص را بر اساس parent_asin پیدا می‌کند و در لایه سرویس با find_one دقیقاً یک Document برمی‌گرداند.

```
coll.find_one({"parent_asin": asin})
```

در Import نهایی، id_ نیز از parent_asin ساخته شده است؛ بنابراین امکان Lookup Point با شناسه بومی Mon-goDB نیز وجود دارد.

۲.۶ Q۲: Pagination کاتالوگ

برای جلوگیری از Load شدن کل Collection، نتایج بر اساس parent_asin مرتب شده و با skip و limit صفحه‌بندی می‌شوند. حداکثر Size Page در کد روی ۱۰۰ محدود شده است.

```
coll.find({}, {"title": 1, "parent_asin": 1, "price": 1}) \
    .sort("parent_asin", 1) \
    .skip((page - 1) * page_size) \
    .limit(page_size)
```

۳.۶ Q۳: جستجوی Category در هر سطح

به دلیل Flatten شدن، ها Query Category مستقیماً وجود رشته Category را در آرایه بررسی می‌کند:

```
coll.find({"categories": category})
```

۴.۶ Q۴: فیلتر یک ویژگی فنی پویا

برای اطمینان از اینکه Key و Value متعلق به یک عضو یکسان از آرایه details هستند، از \$elemMatch استفاده می‌شود:

```
coll.find({
  "details": {
    "$elemMatch": {"k": key, "v": value}
  }
})
```

۵.۶ Q۵: فیلتر چند ویژگی با AND و OR

هر شرط ابتدا به یک \$elemMatch مستقل تبدیل می‌شود. برای حالت AND، تمام شرط‌ها باید برقرار باشند و برای OR وجود حداقل یکی کافی است.

```
clauses = [
  {"details": {"$elemMatch": {"k": c["k"], "v": c["v"]}}}
  for c in conditions
]
filter_and = {"$and": clauses}
filter_or = {"$or": clauses}
```

۷ خروجی Explain برای Queryها

خروجی explain() برای هر Query در دو فاز «کلاستر بدون Index» و «کلاستر بهینه با Index» ذخیره شد. جدول‌های زیر مهم‌ترین فیلدهای executionStats و Plan Winning را مستقیماً از فایل‌های JSON ثبت‌شده نشان می‌دهند.

۱.۷ Index بدون Cluster

Query	Winning stage	nReturned	Keys examined	Docs examined	Time (ms)
Q1	COLLSCAN	1	0	1,610,012	64,146
Q2	SORT + COLLSCAN	20	0	1,610,012	67,058
Q3	COLLSCAN	5,899	0	1,610,012	64,423
Q4	COLLSCAN	52	0	1,610,012	68,404
Q5 AND	COLLSCAN	1	0	1,610,012	57,428
Q5 OR	SUBPLAN + COLLSCAN	53	0	1,610,012	53,599

در نبود Index سفارشی، تمام Queryها مجبور به بررسی کل Document 1,610,012 شده‌اند. Q۲ علاوه بر Scan کامل، مرحله Sort داشته است.

۲.۷ Index با Cluster Optimized

Query	Winning stage	nReturned	Keys examined	Docs examined	Time (ms)
Q1	IXSCAN + FETCH	1	1	1	103
Q2	IXSCAN + FETCH + LIMIT	20	20	20	271
Q3	IXSCAN + FETCH	5,899	5,899	5,899	30,112
Q4	IXSCAN + FETCH	52	52	52	720
Q5 AND	IXSCAN + FETCH	1	1	1	114
Q5 OR	OR + IXSCAN + FETCH	53	53	53	537

بعد از، گذاری Index تعداد های Document بررسی شده تقریباً به تعداد نتایج مورد نیاز کاهش یافته است. برای مثال Q۴ از بررسی Document 1,610,012 به تنها Document 52 رسیده است. در Q۳ با وجود استفاده از Index، چون خود Query باید 5,899 محصول را برگرداند، هزینه Fetch داده‌ها همچنان قابل توجه است و به همین دلیل Speedup آن نسبت به های Query Lookup Point کمتر است.

نکته مربوط به Q۱: فایل Explain ثبت شده برای Q۱ با همان Predicate روی find ذخیره شده و بدون limit(1) است؛ به همین دلیل در فاز بدون Index کل Collection در explain بررسی شده است. Benchmark نهایی Q۱ به صورت جداگانه با رفتار واقعی find_one و ASIN انتهایی B08K7KC4GR اجرا شد تا مقایسه زمان پاسخ منصفانه باشد.

۸ گذاری Index و دلایل انتخاب یا رد فیلدها

چهار Index سفارشی زیر روی Collection ساخته شدند. علاوه بر آن، Index پیش فرض _id_ نیز به طور خودکار توسط MongoDB وجود دارد.

نام Index	نوع	هدف
idx_parent_asin	Ascending Unique	Lookup Point در Q۱ و تضمین یکتایی شناسه محصول
idx_categories	Ascending Multikey	جستجوی Category در آرایه Flatten شده برای Q۳
idx_details_kv	Multikey Compound	فیلتر دقیق زوج k/v در Q۴ و Q۵
idx_title_text	Text	ارزیابی Index Text و جستجوی متنی عنوان محصول

۱.۸ دلیل فنی انتخاب ها Index

- **parent_asin: Q۱** یک Lookup Point پرتکرار و دقیق روی شناسه محصول است. Index یکتا تعداد Docu-ment های بررسی شده را از Collection Scan به یک کلید و یک Document کاهش می دهد و هم زمان Duplicate شدن شناسه منطقی محصول را جلوگیری می کند.
- **categories:** چون این Field آرایه است، Index MongoDB را به صورت Multikey مدیریت می کند و جستجوی membership در هر سطح Category بدون Scan Collection ممکن می شود.
- **details.v: + details.k** های Feature Dynamic Query هم نام ویژگی و هم مقدار آن را نیاز دارند. Com-pound Index مستقیماً با مدل \$elemMatch هم راستا است و Q۴/Q۵ را از Scan کامل دور می کند.
- **text: title** برای بررسی Index Text یک Index متنی روی عنوان انتخاب شد. عنوان کوتاه تر و هدفمندتر از Description است و برای Search متنی محصول مناسب تر است.

۲.۸ دلیل Index نزدن روی سایر ها Field

طبق نیاز پروژه، فقط هایی Index ایجاد شدند که ها Endpoint یا ارزیابی Search Text را مستقیماً پشتیبانی می کنند. Index اضافی باعث مصرف Storage بیشتر، RAM بیشتر و هزینه نگهداری در هر Write می شود. بررسی های Field دیگر به شکل زیر است:

Field	وضعیت	دلیل
_id	Index پیش فرض	MongoDB به صورت خودکار Index Unique روی _id دارد؛ بنابراین ساخت Index سفارشی مجدد توجیهی ندارد.
main_category	بدون Index سفارشی	Query الزامی مستقلی روی این Field نداریم و آرایه categories مسیر چندسطحی را پوشش می دهد.
title	Index Text	برای جستجوی متنی Index شده است؛ Ascending Index جداگانه چون Lookup Sort/Exact الزامی روی Title نداریم ایجاد نشده است.
average_rating	بدون Index	هیچ Query الزامی Range/Sort روی Rating ندارد؛ Index باعث هزینه Write بدون منفعت در Workload فعلی می شود.
rating_number	بدون Index	در های Query پروژه برای Filter یا Sort استفاده نمی شود.
features	بدون Index	آرایه متنی حجیم و متنوع است و Endpoint الزامی روی آن تعریف نشده؛ Index Multikey آن می تواند بسیار بزرگ شود.

Field	وضعیت	دلیل
description	بدون Index	متن طولانی است و Search متنی مورد نیاز با title ارزیابی شده؛ Index Text روی Description حجم و هزینه Update بیشتری ایجاد می‌کند.
price	بدون Index	هیچ Range/Sort Query بر اساس Price در پنچ Query الزامی وجود ندارد.
images	بدون Index	داده Nested مربوط به نمایش محصول است و برای Search/Filter پروژه استفاده نمی‌شود.
videos	بدون Index	مشابه Field Images، محتوایی است و Predicate الزامی روی آن وجود ندارد.
store	بدون Index	Search بر اساس فروشگاه در Workload تعریف نشده و افزودن Index مستقیماً به های Endpoint مورد ارزیابی کمک نمی‌کند.
categories	Index شده	مستقیماً Q3 را پشتیبانی می‌کند.
details	Compound Index روی k/v	Index روی کل Object مناسب نیست؛ مسیرهای details.k و details.v دقیقاً با Predicate مورد استفاده هماهنگ شده‌اند.
parent_asin	Index شده	شناسه اصلی Application برای Q1 است.
bought_together	بدون Index	در های Query پروژه استفاده نمی‌شود و در بسیاری از رکوردها مقدار مفیدی برای Search ندارد.
demo_views	بدون Index	Field آزمایشی و Update-heavy برای Condition Race است؛ Index روی آن فقط هزینه هر Update را افزایش می‌دهد و هیچ Read Query بر اساس آن نداریم.

۹ شبیه‌سازی Condition Race و رفع آن

برای شبیه‌سازی Condition Race، یک Field آزمایشی به نام demo_views روی یک محصول واقعی استفاده شد. در روش Unsafe، هر Worker ابتدا مقدار فعلی را می‌خواند و سپس مقدار $current+1$ را با $$set$ می‌نویسد. اگر چند Worker همزمان مقدار یکسانی را بخوانند، های Write بعدی یکدیگر را Overwrite می‌کنند و Update Lost رخ می‌دهد.

```
# Unsafe read-modify-write
doc = coll.find_one({"_id": product_id}, {"demo_views": 1})
current = doc.get("demo_views", 0)
coll.update_one(
    {"_id": product_id},
    {"$set": {"demo_views": current + 1}}
)
```

راه‌حل انتخابی استفاده از عملیات Atomic تک Document با $$inc$ است:

```
coll.update_one(
```

```
{ "_id": product_id },
  { "$inc": { "demo_views": 1 } }
}
```

MongoDB عملیات Update یک Document را به صورت Atomic انجام می‌دهد؛ بنابراین Increment همزمان بدون نیاز به Read قبلی، Update Lost ایجاد نمی‌کند.

۱.۹ نتایج Condition Race Test Load

در هر آزمایش 500 عملیات Update اجرا شد و تعداد Workerها برابر 10, 25, 50, 100 بود.

Method	Workers	Throughput (req/s)	Final	Expected	Lost updates
Unsafe	10	70.61	118	500	382
Atomic	10	58.60	500	500	0
Unsafe	25	143.01	66	500	434
Atomic	25	120.63	500	500	0
Unsafe	50	225.95	52	500	448
Atomic	50	232.49	500	500	0
Unsafe	100	370.08	46	500	454
Atomic	100	392.55	500	500	0

در تمام حالت‌ها، هر 500 درخواست از دید Acknowledge Driver شده‌اند؛ اما در روش Unsafe مقدار نهایی به شدت کمتر از ۵۰۰ است. بنابراین Request Successful به تنهایی تضمین‌کننده صحت داده نیست. روش Atomic در تمام Loadها دقیقاً مقدار نهایی ۵۰۰ و صفر Update Lost ایجاد کرده است.

۲.۹ روش‌های جایگزین

- **Control: Concurrency Optimistic** می‌توان Version یا مقدار قبلی را در شرط Update قرار داد و در صورت Conflict عملیات را Retry کرد. این روش برای منطق‌های پیچیده مناسب است اما برای Increment ساده، \$inc کوتاه‌تر و کم‌هزینه‌تر است.

- **Transaction: Read و Write** می‌توان در Transaction قرار داد؛ اما برای Update یک Field در یک Document، Transaction سربار بیشتری نسبت به عملیات Atomic داخلی MongoDB دارد.

- **Lock: Distributed** قفل Application-level می‌تواند ها-Writer را Serial کند، ولی Throughput و پیچیدگی سیستم را کاهش می‌دهد و نیاز به مدیریت Timeout و Lock Failure دارد.

برای سناریوی شمارنده بازديد، \$inc انتخاب مناسب‌تری است زیرا بدون Lock خارجی و بدون Transaction چندمرحله‌ای، مستقیماً Atomicity موردنیاز را فراهم می‌کند.

۱۰ ارزیابی Performance در سه حالت

تمام های Query Q۱ تا Q۵ در سه فاز جداگانه با Workload یکسان اجرا شدند. در هر Query یک Warmup و پنج Run اندازه گیری شد و شاخص های، Average، P۵۰، P۹۵، Min و Max ثبت شدند. سه حالت دقیق آزمایش عبارت اند از:

۱. **Index: No Standalone** یک MongoDB مستقل، بدون Set Replica و بدون های Index سفارشی؛

۲. **Index: No Cluster** Set Replica سه نودی فعال با secondaryPreferred و بدون های Index پروژه؛

۳. **Cluster: Optimized** همان Set Replica با چهار Index سفارشی فعال.

برای حالت Standalone، همان Document 1,610,012 از کلاستر به MongoDB مستقل منتقل شدند و های Index سفارشی Restore نشدند تا Dataset در هر سه حالت یکسان باشد.

۱.۱۰ Index No Standalone

Query	Avg (ms)	P50	P95	Min	Max
Q1	9,192.59	6,796.24	16,413.15	6,726.10	18,800.80
Q2	6,891.94	6,872.37	7,104.13	6,577.35	7,110.47
Q3	7,249.28	7,230.88	7,355.45	7,127.50	7,360.34
Q4	8,864.33	8,896.18	9,115.12	8,601.91	9,168.32
Q5 AND	8,760.15	8,750.65	8,873.25	8,635.21	8,885.01
Q5 OR	11,213.35	11,199.04	11,463.31	11,027.54	11,528.02

۲.۱۰ Index No Cluster

Query	Avg (ms)	P50	P95	Min	Max
Q1	37,595.93	39,059.01	72,762.14	7,121.32	76,619.52
Q2	70,280.26	70,112.65	74,312.60	65,697.99	74,939.25
Q3	50,222.50	67,388.28	74,001.07	15,470.83	74,047.14
Q4	38,108.84	37,761.29	69,120.51	11,730.60	72,610.39
Q5 AND	37,876.54	35,288.05	64,687.67	10,300.91	67,407.31
Q5 OR	54,012.89	63,339.64	64,873.19	18,633.59	64,976.82

بدون Index، های Query کلاستر برای بسیاری از اجراها نیازمند Scan گسترده داده روی ها Replicand و در محیط آزمایش نوسان I/O و Cache قابل توجهی مشاهده شده است. این اعداد نتیجه اندازه گیری همین محیط هستند و نباید به

عنوان یک قانون کلی مبنی بر کندتر بودن Set Replica از Standalone تفسیر شوند؛ عامل اصلی مشترک در این فاز، نبود Index و هزینه کامل Scan Collection است.

۳.۱۰ Cluster Optimized

Query	Avg (ms)	P50	P95	Min	Max
Q1	62.12	1.01	245.45	0.82	306.47
Q2	53.44	1.59	209.03	1.37	260.83
Q3	7,816.92	488.36	29,916.55	264.69	37,209.57
Q4	7.35	4.07	17.37	2.87	20.02
Q5 AND	2.68	1.36	6.89	1.08	8.24
Q5 OR	3.03	3.08	3.20	2.86	3.23

در حالت بهینه، Q۱، Q۲، Q۴ و Q۵ معمولاً در مقیاس چند میلی ثانیه پاسخ می‌دهند. در Q۱ چهار Run عادی نزدیک یک میلی ثانیه بودند و یک Run حدود 306 ms باعث افزایش Average به 62.12 ms شد؛ به همین دلیل P۵۰ برابر 1.01 ms رفتار معمول Query را بهتر نشان می‌دهد.

Q۳ نیز نوسان زیادی دارد: P۵۰ حدود 488 ms است اما یک Run تا حدود 37.2 s افزایش یافته و Average را به 7.82 s رسانده است. علاوه بر نوسان، Q۳ Cache/I/O باید هزاران Document واقعی را Fetch کند و به همین دلیل حتی با IXSCAN هزینه آن بیشتر از Point Lookup است.

۴.۱۰ مقایسه اثر Index با Explain

مقایسه مستقیم executionTimeMillis در های Explain قبل و بعد از Index نشان می‌دهد مسیر اجرای Query به طور اساسی تغییر کرده است. برای نمونه:

• Q۱: از 64,146 ms با COLLSCAN به 103 ms با IXSCAN؛

• Q۲: از 67,058 ms به 271 ms؛

• Q۴: از 68,404 ms به 720 ms؛

• Q۵ AND: از 57,428 ms به 114 ms؛

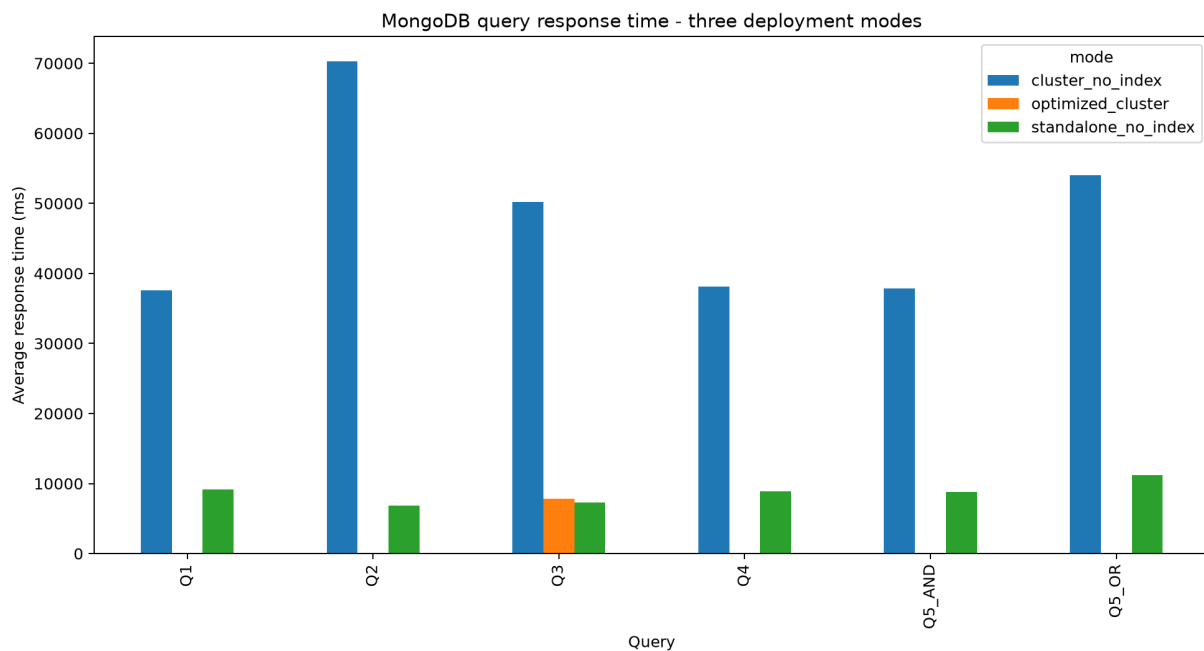
• Q۵ OR: از 53,599 ms به 537 ms.

این کاهش مستقیماً با افت totalDocsExamined از کل Collection به تعداد بسیار محدود Documentها همزمان است و نشان می‌دهد بهبود اصلی ناشی از انتخاب مسیر IXSCAN به جای COLLSCAN است.

۱۱ نمودارهای نتایج

نمودارهای زیر مستقیماً از فایل‌های آماری نهایی توسط اسکریپت Python و کتابخانه Matplotlib/Pandas ساخته شده‌اند.

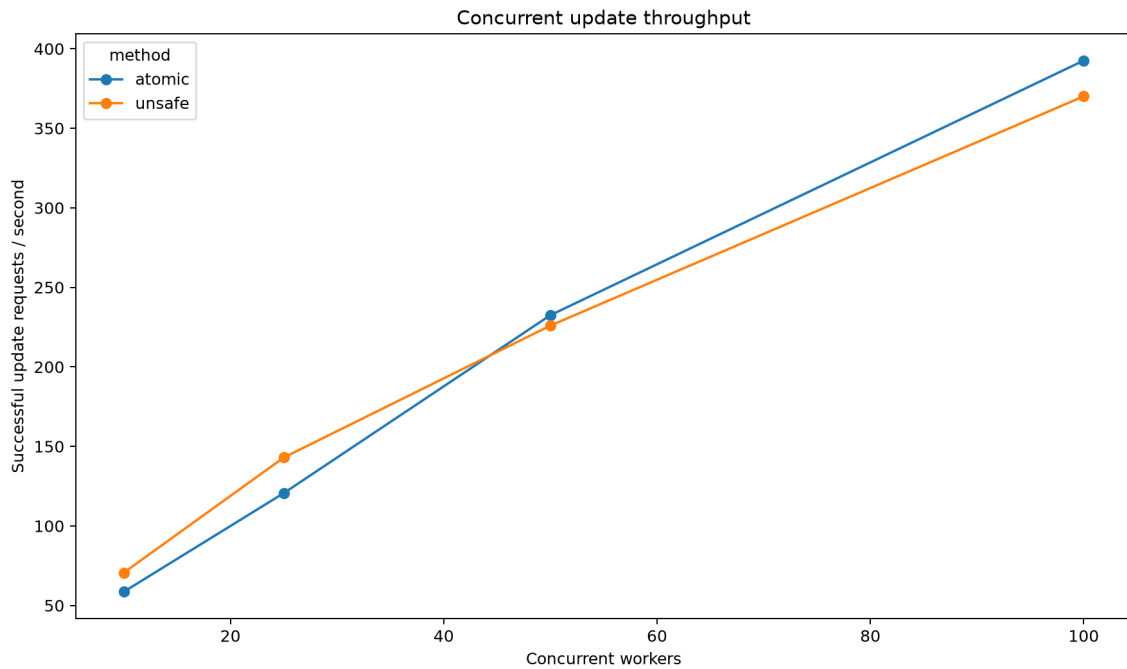
۱.۱۱ نمودار مقایسه زمان پاسخ Queryها در سه حالت



شکل ۱: مقایسه Time Response Average برای Queryها در سه Mode Deployment

در نمودار زمان پاسخ، حالت Cluster بدون Index بیشترین زمان‌ها را برای اغلب Queryها نشان می‌دهد، زیرا Scan کامل Collection انجام می‌شود. حالت Standalone بدون Index نیز چند ثانیه برای هر Query زمان نیاز دارد. پس از اعمال، ها، Index زمان، Q1، Q2، Q4 و Q5 به طور چشمگیری کاهش یافته است. Q3 همچنان از بقیه سنگین‌تر است زیرا تعداد خروجی آن زیاد است و Fetch هزاران Document حتی پس از یافتن کلیدهای Index لازم است. استفاده هم‌زمان از جدول‌های P50/P95 در کنار Average برای تشخیص‌های Outlier مشاهده‌شده در Q1 و Q3 ضروری است.

۲.۱۱ نمودار Throughput در تست Condition Race



شکل ۲: تعداد درخواست Update موفق در ثانیه تحت بار همزمان

با افزایش تعداد، ها،Throughput Worker هر دو روش افزایش یافته است. نکته اصلی این نمودار آن است که روش Unsafe می‌تواند Throughput ظاهراً مناسب و حتی ۵۰۰ درخواست Acknowledged داشته باشد، اما داده نهایی نادرست است. روش Atomic در تمام ها،Load صفر Update Lost دارد و در بارهای ۵۰ و ۱۰۰ Worker حتی Throughput آن از روش Unsafe بیشتر شده است. بنابراین در ارزیابی Update همزمان، صحت داده در کنار تعداد Request موفق معیار اصلی انتخاب روش است.